PCCST403 OPERATING SYSTEMS

MODULE 4

File Concept

- Computers can store information on various storage media.
- The operating system abstracts from the physical properties of its storage devices to define a logical storage unit called a **file.**
- Files are **mapped** by the operating system onto non-volatile physical devices.
- A file is a **named collection** of **related information** that is recorded on secondary storage.
- For a user's perspective, a file is the **smallest allotment** of logical secondary storage.
- Files commonly represent programs (both source and object code) and data.
- Data file contents may be numeric, alphabetic, alphanumeric, or binary
- Files may be **free form** (e.g., text files) or **rigidly formatted**
- In general, a file is a **sequence** of bits, bytes, lines, or records, whose meaning is defined by the file's creator and user .
- A file has a certain defined structure, which depends on its type.
- A **text file is a sequence of characters organized** into lines (and possibly pages).
- A **source file is a sequence of functions, each of** which is further organized as declarations followed by executable statements.
- An **executable file is a series of code sections that the loader can bring into** memory and execute.

File Attributes

A file's attributes vary from one operating system to another but typically consist of these:

- **Name**. The symbolic file name is the only information kept in human readable form.
- **Identifier.** This unique tag, usually a number, identifies the file within the file system; it is the non-human-readable name for the file.
- **Type.** This information is needed for systems that support different types of files.
- **Location.** This information is a pointer to a device and to the location of the file on that device.
- **Size.** The current size of the file (in bytes, words, or blocks) and possibly the maximum allowed size are included in this attribute.
- **Protection.** Access-control information determines who can do reading, writing, executing, and so on.
- **Time, date, and user identification.** This information may be kept for creation, last modification, and last use. These data can be useful for protection, security, and usage monitoring.

File Operations

The operating system can provide system calls to create, write, read, reposition, delete, and truncate files.

1. **Creating a file.** File is created with no data.

2. **Writing a file.** To write a file, make a system call specifying both the name of the file and the information to be written to the file. Given the name of the file, the system searches the directory to find the file's location. The system must keep a write pointer to the location in the file where the next write is to take place. The write pointer must be updated whenever a write occurs.
3. **Reading a file.** To read from a file, use a system call that specifies the name of the file and where (in memory) the next block of the file should be put. The directory is searched for the associated entry, and the system needs to keep a read pointer to the location in the file where the next read is to take place. Once the read has taken place, the read pointer is updated.
4. **Repositioning within a file.** The current-file-position pointer is repositioned to a given value. This file operation is also known as a file seek . Used for random access.
5. **Deleting a file.** To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space, so that it can be reused by other files, and erase the directory entry.
6. **Truncating a file.** The user may want to erase the contents of a file but keep its attributes. This function allows all attributes to remain unchanged—except for file length—but lets the file be reset to length zero and its file space released.

File Type

File type refers to the ability of the operating system to distinguish different types of file such as text files source files and binary files etc. Many operating systems support many types of files. Operating system like MS-DOS and UNIX have the following types of files –

Ordinary files

- These are the files that contain user information.
- These may have text, databases or executable program.
- The user can apply various operations on such files like add, modify, delete or even remove the entire file.

Directory files

- These files contain list of file names and other information related to these files.

Special files

- These files are also known as device files.
- These files represent physical device like disks, terminals, printers, networks, tape drive etc.

These files are of two types –

- **Character special files** – data is handled character by character as in case of terminals or printers.
- **Block special files** – data is handled in blocks as in the case of disks and tapes.

OPERATING SYSTEMS

File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

File Structure

A file has various kinds of structure. Some of them can be :

- **Simple Record Structure** with lines of fixed or variable lengths.
- **Complex Structures** like formatted document or reloadable load files.
- **No Definite Structure** like sequence of words and bytes etc.
- Who decides:
  - Operating system
  - Program
  - Disadvantages of having the operating system support multiple file structures
  - Some operating systems impose (and support) a minimal number of file structures.

File Control Block (FCB)

OS maintains FCB per file, which contains many details about the file

Typically, inode number, permissions, size, dates

Example

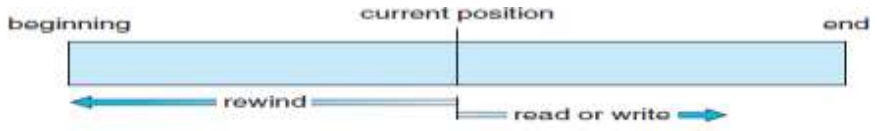
file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

File Access Methods

- Files store information. When it is used, this information must be accessed and read into computer memory.
- The information in the file can be accessed in several ways.

1) Sequential Access

- The simplest access method is sequential access.
- Information in the file is processed in order, one record after the other.
- Editors and compilers usually access files in this fashion.
- A read operation—**read next()**—reads the next portion of the file and automatically advances a file pointer.
- The write operation—**write next()**—appends to the end of the file and advances to the end of the newly written material .



Sequential-access file.

2) Direct Access

- Here, a file is made up of fixed-length logical records that allow programs to read and write records rapidly in no particular order.
- For direct access, the file is viewed as a numbered sequence of blocks or records.
- We may read block 14, then read block 53, and then write block 7.
- There are no restrictions on the order of reading or writing for a direct-access file.
- Databases are often direct access .
- When a query concerning a particular subject arrives, we compute which block contains the answer and then read that block directly to provide the desired information.
- Read operation for direct access is read(n), where *n is the block number*
- *Write* operation for direct access is *write(n)*

3) Other Access Methods

- Other access methods can be built on top of a direct-access method.
- These methods generally involve the construction of an index for the file.
- The index, contains pointers to the various blocks.
- To find a record in the file, we first search the index and then use the pointer to access the file directly and to find the desired record.
- With large files, the index file itself may become too large to be kept in memory.
- One solution is to create an index for the index file.
- The primary index file contains pointers to secondary index files, which point to the actual data items.

- You must adhere to **file-naming conventions** when saving files :

1. *Case sensitivity* – upper and lower case are different
   - True in Linux and Unix variations, not in Windows
2. Maximum *length* (Windows 260 characters)
3. *Spaces* allowed
4. *Digits* allowed
5. \ / : * ? " < > | not allowed *(Special Characters)*
6. File names not allowed (con, nul, prn)

- **File extensions** provide clues to the file contents.
- OS uses extensions to know which application created the file and the internal format of the file.

In this approach, password is associated with each file.
Just as a password is required to access a computer system, access to each file will be also controlled by a password.

A password is a string of characters used to verify the identity of a user during the authentication process. Passwords are typically used in conjuncture with a username; they are designed to be known only to the user and allow that user to gain access to a device, application or website.

Protection in File System

In computer systems, a lot of user's information is stored, the objective of the operating system is to keep safe the data of the user from the improper access to the system. Protection can be provided in number of ways. For a single laptop system, we might provide protection by locking the computer in a desk drawer or file cabinet. For multi-user systems, different mechanisms are used for the protection.

Types of Access :

The files which have direct access of the any user have the need of protection. The files which are not accessible to other users doesn't require any kind of protection. The mechanism of the protection provide the facility of the controlled access by just limiting the types of access to the file. Access can be given or not given to any user depends on several factors, one of which is the type of access required. Several different types of operations can be controlled:

- **Read –** Reading from a file.
- **Write –** Writing or rewriting the file.
- **Execute –** Loading the file and after loading the execution process starts.
- **Append –** Writing the new information to the already existing file, editing must be end at the end of the existing file.
- **Delete –** Deleting the file which is of no use and using its space for the another data.
- **List –** List the name and attributes of the file.

Operations like renaming, editing the existing file, copying; these can also be controlled. There are many protection mechanism. each of them mechanism have different advantages and disadvantages and must be appropriate for the intended application.

Access Control

It can be of two types:

1. **Access List**
2. **Access Group**

There are different methods used by different users to access any file. The general way of protection is to associate *identity-dependent access* with all the files and directories in list called access-control list (ACL) which specify the names of the users and the types of access associate with each of the user. The

main problem with the access list is their length. If we want to allow everyone to read a file, we must list all the users with the read access. This technique has two undesirable consequences:
Constructing such a list may be tedious and unrewarding task, especially if we do not know in advance the list of the users in the system.

Previously, the entry of the any directory is of the fixed size but now it changes to the variable size which results in the complicates space management. These problems can be resolved by use of a condensed version of the access list. To condense the length of the access-control list, many systems recognize three classification of users in connection with each file:

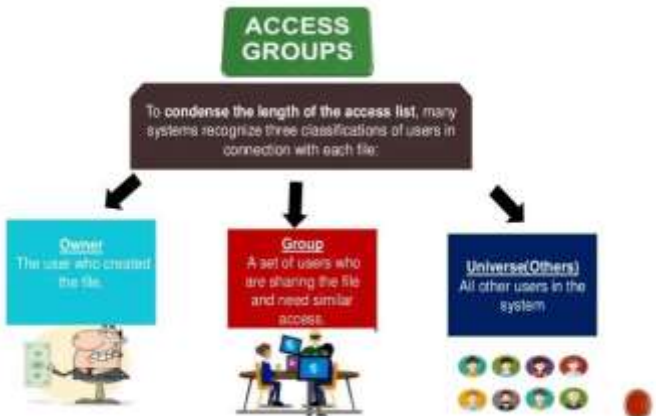
- **Owner –** Owner is the user who has created the file.
- **Group –** A group is a set of members who has similar needs and they are sharing the same file.
- **Universe –** In the system, all other users are under the category called universe.

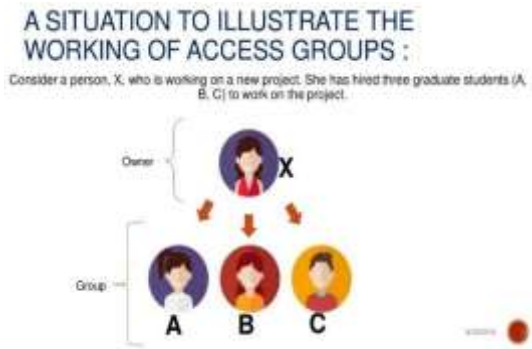
The most common recent approach is to combine access-control lists with the normal general owner, group, and universe access control scheme. For example: Solaris uses the three categories of access by default but allows access-control lists to be added to specific files and directories when more fine-grained access control is desired.

Other Protection Approaches:
The access to any system is also controlled by the password. If the use of password is random and it is changed often, this may be result in limit the effective access to a file.

The use of passwords has a few disadvantages:

- The number of passwords are very large so it is difficult to remember the large passwords.
- If one password is used for all the files, then once it is discovered, all files are accessible; protection is on all-or-none basis.





- All **other users** should be able to execute the file.
  (X is interested in letting as many people as possible play the game in order to obtain appropriate feedback.)
- To achieve such a protection, a new group must be created with member A, B, C. The name of the group must be then associated with the file project, and the access-right must be set in accordance with the policy that is outlined .

Goals of Protection

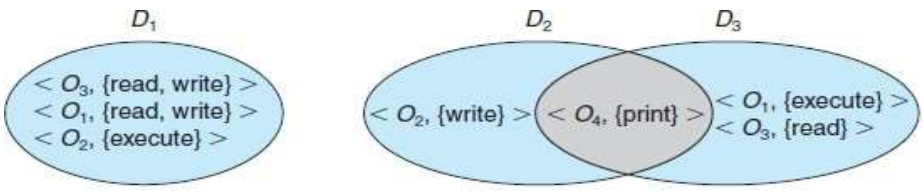
- Prevent the mischievous, intentional violation of an access restriction by a user.
- Need to ensure that each program component active in a system uses system resources only in ways consistent with stated policies.
- An unprotected resource cannot defend against use (or misuse) by an unauthorized or incompetent user.
- The role of protection in a computer system is to provide a mechanism for the enforcement of the policies governing resource use.
- These policies can be established in a variety of ways.
    - Some are fixed in the design of the system, while others are formulated by the management of a system.
    - Some are formulated by the individual users to protect their own files and programs.
- A protection system must have the flexibility to enforce a variety of policies.
- Policies for resource use may vary by application, and they may change over time.
- *Mechanisms are distinct from policies.*
- *Mechanisms determine how something will be done; policies decide what will be done.*
- Policies are likely to change from place to place or time to time.
  In the worst case, every change in policy would require a change in the underlying mechanism.

Domain of Protection

- A computer system is a collection of processes and objects.
- *Objects, means* both hardware objects (such as the CPU, memory segments, printers, disks, and tape drives) and software objects (such as files, programs, and semaphores).
- Each object has a unique name that differentiates it from all other objects in the system, and each can be accessed only through well-defined and meaningful operations.
- A process should be allowed to access only those resources for which it has authorization.

Domain Structure

- A domain is a collection of access rights, each of which is an ordered pair <object-name, rights-set>.
- For example, if domain *D has the access* right < file *F, {read, write}>, then a process executing in domain D can both* read and write file *F. It cannot, however, perform any other operation on that* object.



System with three protection domains.

- A domain can be realized in a variety of ways:
    - Each *user may be a domain. In this case, the set of objects that can be* accessed depends on the identity of the user. Domain switching occurs when the user is changed—generally when one user logs out and another user logs in.

- Each *process may be a domain. In this case, the set of objects that can be* accessed depends on the identity of the process. Domain switching occurs when one process sends a message to another process and then waits for a response.
- Each *procedure may be a domain. In this case, the set of objects that can be* accessed corresponds to the local variables defined within the procedure. Domain switching occurs when a procedure call is made.

Access Matrix

- General model of protection can be viewed abstractly as a matrix, called an access matrix.
- The rows of the access matrix represent domains, and the columns represent objects.
- Each entry in the matrix consists of a set of access rights.
- The entry access(*i,j) defines the set of operations* that a process executing in domain *Di can invoke on object Oj .*

domain \ object	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Access matrix.

- There are four domains and four objects—three files (*F1, F2, F3) and one* laser printer.
- A process executing in domain *D1 can read files F1 and F3.*
- *A* process executing in domain *D4 has the same privileges as one executing in* domain *D1; but in addition, it can also write onto files F1 and F3.*
- *The laser* printer can be accessed only by a process executing in domain *D2.*
- Processes should be able to switch from one domain to another.

- Switching from domain *Di to domain Dj is allowed if and only if the access right switch* ∈ access(*i, j).*

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

-

Process executing in domain *D2 can switch* to domain *D3 or to domain D4. A process in domain D4 can switch to D1, and* one in domain *D1 can switch to D2.*

- The ability to copy an access right from one domain (or row) of the access matrix to another is denoted by an asterisk (*) appended to the access right.
- The copy right allows the access right to be copied only within the column (that is, for the object) for which the right is defined.

- In the Figure (a), a process executing in domain *D2 can copy the read operation into any* entry associated with file *F2*

domain \ object	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

domain \ object	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

(b)

Access matrix with ***copy*** rights.

- Addition of new rights and removal of some rights. The owner right controls these operations.
- If access(*i, j) includes* the owner right, then a process executing in domain *Di can add and remove* any right in any entry in column *j.*

- Figure (a), domain *D1* is the owner of *F1 and thus can add and delete any valid right in column F1.*
- Similarly, domain *D2 is the owner of F2 and F3 and thus can add and remove* any valid right within these two columns.
- Thus, the access matrix of Figure (a) can be modified to the access matrix shown in Figure (b).

domain \ object	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		

(a)

domain \ object	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write

(b)

Access matrix with owner rights.

- The copy and owner rights allow a process to change the entries in a column.

- The control right is applicable only to domain objects. If access(*i, j) includes the* control right, then a process executing in domain *Di can remove any access* right from row *j.*

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

Fig(a)

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch control
D_3		read	execute					
D_4	write		write		switch			

Modified access matrix of fig(a)

ALLOCATION METHODS

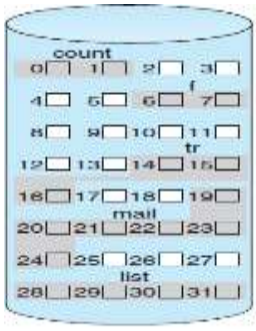
1) Contiguous Allocation

2) Linked Allocation

3) Indexed Allocation

1. Allocation Methods- Contiguous Allocation

- Contiguous allocation requires that each file occupy a set of contiguous blocks on the disk.
- Contiguous allocation of a file is defined by the disk address and length (in block units) of the first block.
- If the file is *n blocks long and starts at location b, then it occupies blocks b, b + 1, b + 2, ..., b + n − 1.*
- *The directory entry for* each file indicates the address of the starting block and the length of the area allocated for this file



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Contiguous allocation of disk space.

- Both sequential and direct access can be supported by contiguous allocation.
- **Problem**
- **External fragmentation.**
- Solution
- Copy an entire file system onto another disk.

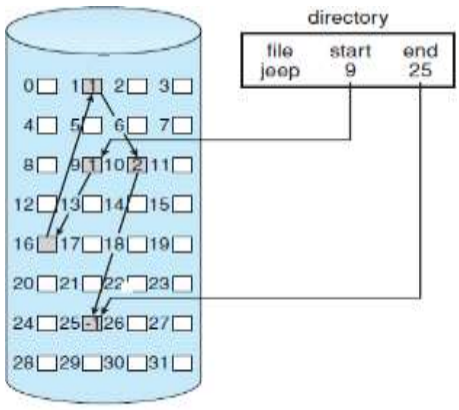
- The disk is then freed completely, creating one large contiguous free space.
- Then copy the files back onto the original disk by allocating contiguous space from this one large hole.
- This scheme effectively compacts all free space into one contiguous space, solving the fragmentation problem.
- Another problem with contiguous allocation is determining how much space is needed for a file.
- If we allocate too little space to a file, we may find that the file cannot be extended.
- A contiguous chunk of space is allocated initially. Then, if that amount proves not to be large enough, another chunk of contiguous space, known as an extent, is added.
- The location of a file's blocks is then recorded as a location and a block count, plus a link to the first block of the next extent.

2. Allocation Methods-Linked Allocation

- Linked allocation solves all problems of contiguous allocation.
- With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk.
- The directory contains a pointer to the first and last blocks of the file.
- Eg: A file of five blocks might start at block 9 and continue at block 16, then block 1, then block 10, and finally block 25.
- A write to the file causes the free-space management system to find a free block, and this new block is written to and is linked to the end of the file.
- To read a file, we simply read blocks by following the pointers from block to block.

There is no external fragmentation with linked allocation, and any free block on the free-space list can be used to satisfy a request.

- The size of a file need not be declared when the file is created. A file can continue to grow as long as free blocks are available.



Linked allocation of disk space.

Disadvantages

1) It can be used effectively only for sequential-access files.

- To find the *i^{th} block of a file, we must start at the beginning of that file and follow the* pointers until we get to the *i^{th} block.*

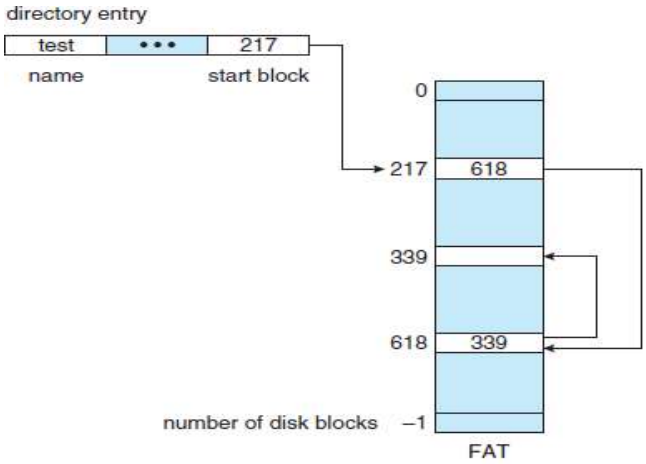
***2)* Space requirement for the pointers.**

- If a pointer requires 4 bytes out of a 512-byte block, then 0.78 percent of the disk is being used for pointers, rather than for information.
- Solution to this problem is to collect blocks into multiples, called clusters, and to allocate clusters rather than blocks

3) Not Reliable.

- A bug in the operating-system software or a disk hardware failure might result in picking up the wrong pointer.
- This error could in turn result in linking into the free-space list or into another file.
- Solutions

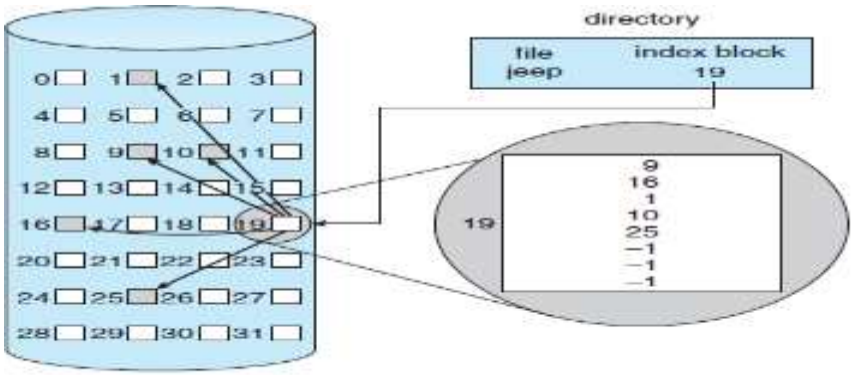
- Use doubly linked lists.
- Store the file name and relative block number in each block.
- Variation on linked allocation is the use of a file-allocation table (FAT).
- File-allocation table (FAT)
- A section of disk at the beginning of each volume is set aside to contain the table.
- The table has one entry for each disk block and is indexed by block number.
- The directory entry contains the block number of the first block of the file.
- The table entry indexed by that block number contains the block number of the next block in the file.



File-allocation table.

- This chain continues until it reaches the last block, which has a special end-of-file value as the table entry.
- An unused block is indicated by a table value of 0.

3. Allocation Methods-Indexed Allocation



Indexed allocation of disk space.

- Each file has its own index block, which is an array of disk-block addresses.
- The *ith entry in the index block points to the ith block of the file.*
- *The directory* contains the address of the index.
- To find and read the *ith* block, we use the pointer in the *ith index-block entry.*
- When the file is created, all pointers in the index block are set to null.
- When the *ith block is first written, a block is obtained from the free-space* manager, and its address is put in the *ith index-block entry.*

- Indexed allocation supports direct access, without suffering from external fragmentation.

Disadvantage

- The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.

Linked scheme.

- An index block is normally one disk block.
- Thus, it can be read and written directly by itself.
- To allow for large files, we can link together several index blocks.

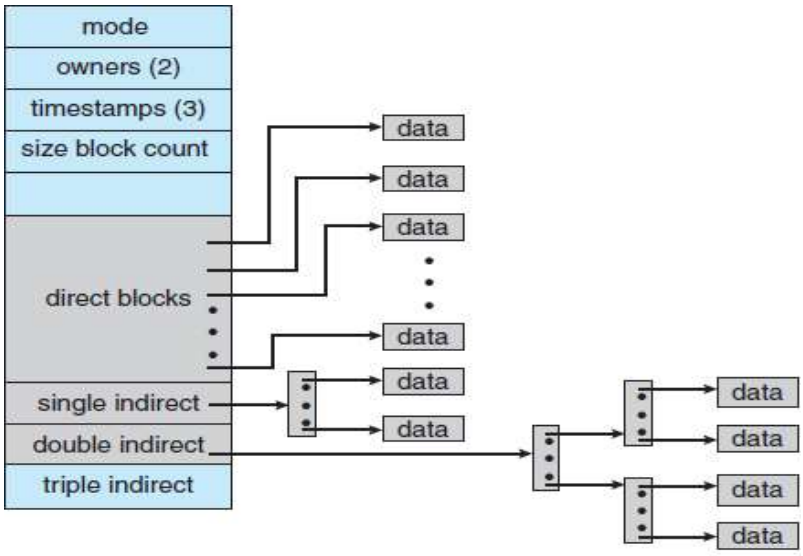
Multilevel index.

- A variant of linked representation uses a first-level index block to point to a set of second-level index blocks, which in turn point to the file blocks.
- To access a block, the operating system uses the first-level index to find a second-level index block and then uses that block to find the desired data block

Combined scheme.

- Keep the first, say, 15 pointers of the index block in the file's inode.
- The first 12 of these pointers point to direct blocks; that is, they contain addresses of blocks that contain data of the file.
- Thus, the data for small files (of no more than 12 blocks) do not need a separate index block.
- If the block size is 4 KB, then up to 48 KB of data can be accessed directly.
- The next three pointers point to indirect blocks.
- The first points to a single indirect block, which is an index block containing not data but the addresses of blocks that do contain data.

- The second points to a double indirect block, which contains the address of a block that contains the addresses of blocks that contain pointers to the actual data blocks.
- The last pointer contains the address of a triple indirect block.



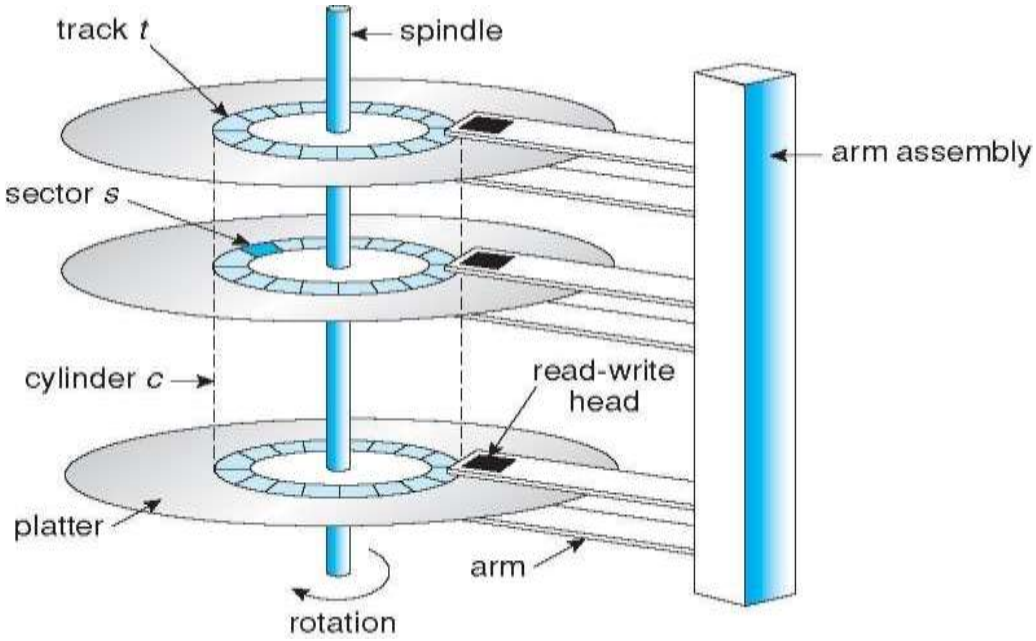
The UNIX inode.

STORAGE MANAGEMENT

Overview of Mass-Storage Structure

1) Magnetic Disks

- **Magnetic disks provide the bulk of secondary storage for modern computer systems.**



- Each disk platter has a flat circular shape.
- A read–write head "flies" just above each surface of every platter.
- The heads are attached to a disk arm that moves all the heads as a unit.

- The surface of a platter is logically divided into circular tracks, which are subdivided into sectors.
- The set of tracks that are at one arm position makes up a cylinder.
- There may be thousands of concentric cylinders in a disk drive, and each track may contain hundreds of sectors.
- The storage capacity of common disk drives is measured in gigabytes.
- Modern magnetic disk drives are addressed as large one-dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer.
- The size of a logical block is usually 512 bytes, although some disks can be low-level formatted to have a different logical block size, such as 1,024 bytes.
- The one-dimensional array of logical blocks is mapped onto the sectors of the disk sequentially.
- Sector 0 is the first sector of the first track on the outermost cylinder.

The mapping proceeds in order through that track, then through the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.

2) Magnetic tape

- Was early secondary-storage medium
- Relatively permanent and holds large quantities of data
- Access time slow
- Random access ~1000 times slower than disk
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems
- Kept in spool and wound or rewound past read-write head
- 20-200GB typical storage
- Common technologies are 4mm, 8mm, 19mm, LTO-2 and SDLT

Hard Disk Drives

- Platters range from .85” to 14” (historically)
- Commonly 3.5”, 2.5”, and 1.8”
- Range from 30GB to 3TB per drive
- Performance
  - Transfer Rate – theoretical – 6 Gb/sec
  - Effective Transfer Rate – real – 1Gb/sec
- Seek time from 3ms to 12ms – 9ms common for desktop drives
- Average seek time measured or calculated based on 1/3 of tracks
- Latency based on spindle speed

1 / (RPM / 60) = 60 / RPM

Average latency = ½ latency

Nonvolatile Memory Devices

A solid-state drive (SSD) is a solid-state storage device that uses integrated circuit assemblies to store data persistently, typically using flash memory, and functioning as secondary storage in the hierarchy of computer storage.

In comparison to hard disk drives and similar electromechanical media which use moving parts, SSDs are typically more resistant to physical shock, run silently, and have higher input/output rates and lower latency. Other forms include USB drives (thumb drive, flash drive), DRAM(Dynamic random-access memory) disk replacements, surface-mounted on motherboards, and main storage in devices like smartphones

- Can be more reliable than HDDs
- More expensive per MB
- Maybe have shorter life span – need careful management

TERMS RELATED TO DISKS

- **Seek Time :** Seek time is the time taken in locating the disk arm to a specified track where the read/write request will be satisfied.
- **Rotational Latency :** It is the time taken by the desired sector to rotate itself to the position from where it can access the R/W heads.
- **Transfer Time :** It is the time taken to transfer the data.
- **Disk Access Time :** Disk access time is given as,

Disk Access Time = Rotational Latency + Seek Time + Transfer Time

- **Disk Response Time :** It is the average of time spent by each request waiting for the IO operation.

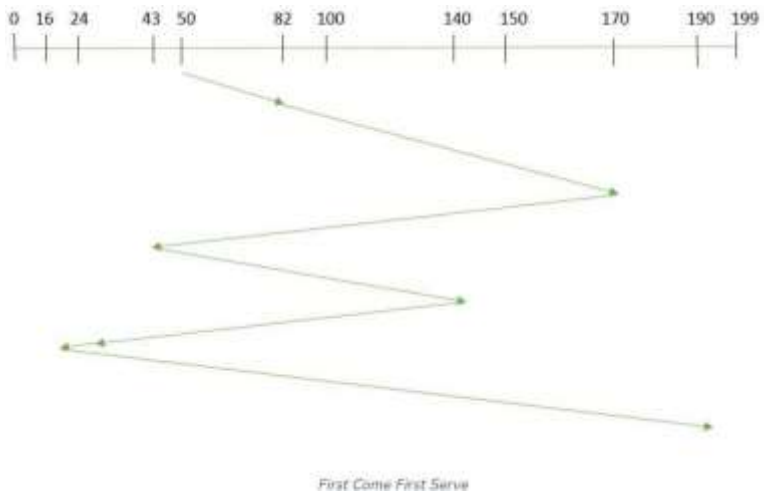
DISK SCHEDULING

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth
- Minimize seek time
- Disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.
- Process needs CPU time and I/O time. For I/O it requests OS to access the disks.
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer. OS maintains queue of requests, per disk or device
- Idle disk can immediately work on I/O request, busy disk means work must queue
    - Optimization algorithms only make sense when a queue exists

The technique that operating system uses to determine the request which is to be satisfied next is called disk scheduling.

1. FCFS Scheduling

- Simplest form of disk scheduling is First come First served.
- It services the I/O requests in the order in which they arrive.



Example:

Suppose the order of request is- (82,170,43,140,24,16,190)

And current position of Read/Write head is: 50

So, total overhead movement (total distance covered by the disk arm) =
(82-50)+(170-82)+(170-43)+(140-43)+(140-24)+(24-16)+(190-16) =642

Advantages of FCFS

Here are some of the advantages of First Come First Serve.

- Every request gets a fair chance
- No indefinite postponement

Disadvantages of FCFS

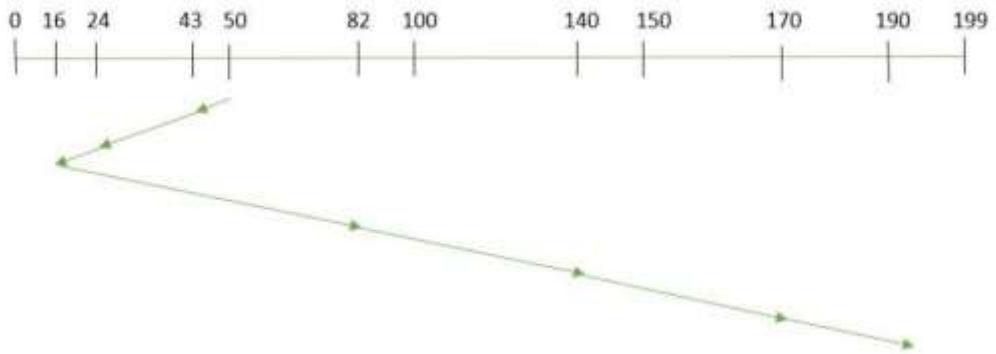
Here are some of the disadvantages of First Come First Serve.

- Does not try to optimize seek time
- May not provide the best possible service

2. SSTF Scheduling-Shortest Seek Time First

- Selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests

Example:



Shortest Seek Time First

Suppose the order of request is- (82,170,43,140,24,16,190)
And current position of Read/Write head is: 50
So,

total overhead movement (total distance covered by the disk arm) =
(50-43)+(43-24)+(24-16)+(82-16)+(140-82)+(170-140)+(190-170) =208

Advantages of Shortest Seek Time First

Here are some of the advantages of Shortest Seek Time First.

- The average Response Time decreases
- Throughput increases

Disadvantages of Shortest Seek Time First

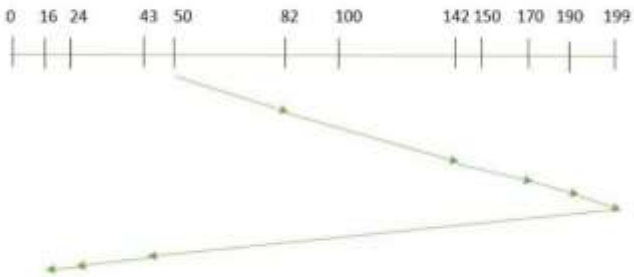
Here are some of the disadvantages of Shortest Seek Time First.

- Overhead to calculate seek time in advance
- Can cause Starvation for a request if it has a higher seek time as compared to incoming requests
- The high variance of response time as SSTF favours only some requests.

3. SCAN

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- SCAN algorithm Sometimes called the elevator algorithm
- But note that if requests are uniformly dense, largest density at other end of disk and those wait the longest.

Example:



SCAN Algorithm

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as

= (199-50) + (199-16) = 332

Advantages of SCAN Algorithm

Here are some of the advantages of the SCAN Algorithm.

- High throughput
- Low variance of response time
- Average response time

Disadvantages of SCAN Algorithm

Here are some of the disadvantages of the SCAN Algorithm.

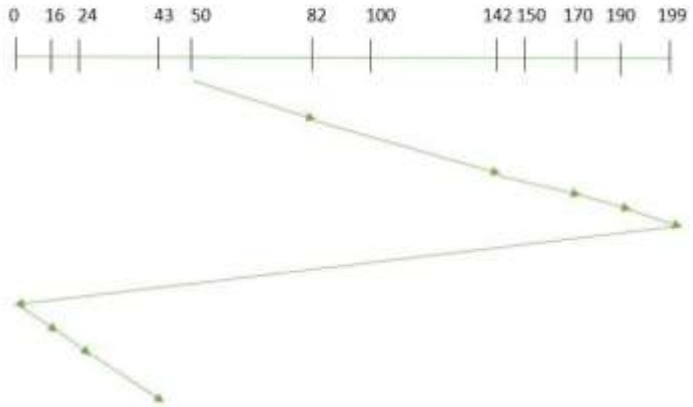
- Long waiting time for requests for locations just visited by disk arm

4. C-SCAN

In the SCAN algorithm, the disk arm again scans the path that has been scanned, after reversing its direction. So, it may be possible that too many requests are waiting at the other end or there may be zero or few requests pending at the scanned area.

These situations are avoided in the *CSCAN* algorithm in which the disk arm instead of reversing its direction goes to the other end of the disk and starts servicing the requests from there. So, the disk arm moves in a circular fashion and this algorithm is also similar to the SCAN algorithm hence it is known as C-SCAN (Circular SCAN).

Example:



Circular SCAN

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

So, the total overhead movement (total distance covered by the disk arm) is calculated as:

=(199-50) + (199-0) + (43-0) = 391

Advantages of C-SCAN Algorithm

Here are some of the advantages of C-SCAN.

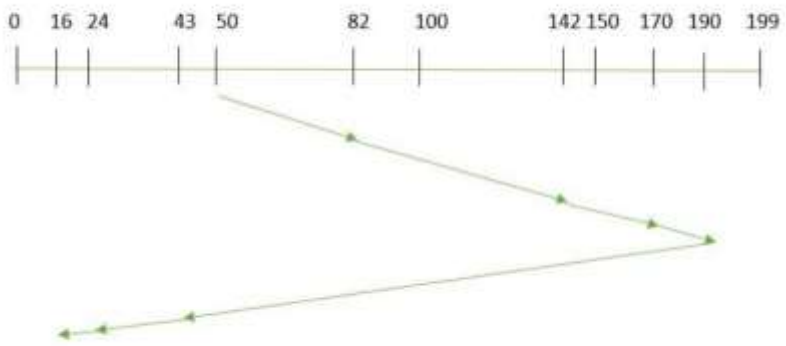
- Provides more uniform wait time compared to SCAN.

5. LOOK Scheduling

LOOK Algorithm is similar to the SCAN disk scheduling algorithm except for the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head

and then reverses its direction from there only. Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Example:



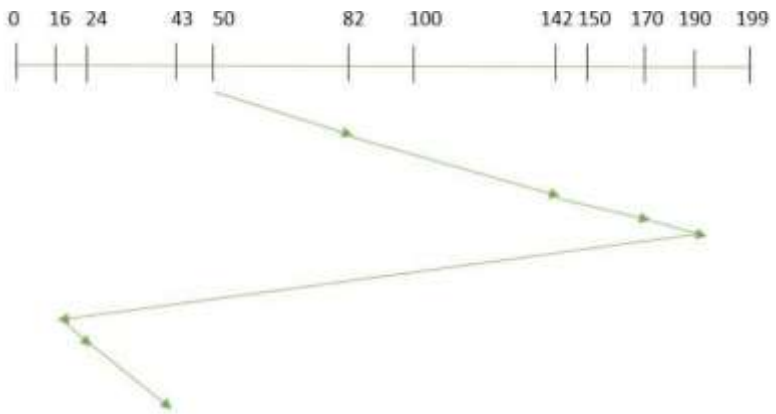
LOOK Algorithm

Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**
So, the total overhead movement (total distance covered by the disk arm) is calculated as:
= (190-50) + (190-16) = 314

6. C-LOOK Scheduling

As LOOK is similar to the SCAN algorithm, in a similar way, C-LOOK is similar to the CSCAN disk scheduling algorithm. In CLOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.
Example:

1. Suppose the requests to be addressed are-82,170,43,140,24,16,190. And the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value"**



C-LOOK

So, the total overhead movement (total distance covered by the disk arm) is calculated as
= (190-50) + (190-16) + (43-16) = 341
****